A-MYCO: Prototyping the Eyes of a Myco-Robot

Chiara D'Amato
MSc in Engineering Physics
Politecnico di Milano
Vittuone, Italy
chiara.damato@mail.polimi.it

Edoardo Torre
MSc in Engineering Physics
Politecnico di Milano
Merate, Italy
edoardo.torre@mail.polimi.it

Lorenzo Vergata
MSc in Engineering Physics
Politecnico di Milano
Milan, Italy
lorenzo.vergata@mail.polimi.it

***Abstract*—This research aims to find a solution that helps hikers and encourages mushroom enthusiasts to take walks in the woods in order to improve their health. The main goal of the project is to provide smart experimental support allowing people to focus solely on the risks of their journey while leaving the real-time detection and classification of mushrooms in the surrounding area to A-MYCO.**
Thanks to access to the most organised mushroom image libraries available, a neural network model performs inference to detect and classify the mushrooms present in the scene. The device alerts the user only when a mushroom species appears, allowing them to enjoy their walk without distractions.
This work is to be understood as an investigation of the potential of a portable and inexpensive device, or a robot, in the field of mushroom object detection and classification, and as such it does not want to replace in any way the work of a mycologist. The results obtained are promising for the development of a compact device tailored to these objectives.

***Index Terms*—classification, detection, dataset, mushroom.**

I. Introduction

The events of COVID-19 have made people feel more comfortable staying home, causing a deterioration in general mental well-being, especially for those living in large concrete cities. Taking a walk in the forest is highly beneficial for health and helps in stress reduction. The idea behind **A-MYCO** is *to leverage people's interest in mushrooms* and *to stimulate them to take forest walks in a safer way*.
In the year 2023, data provided by the Alpine and Speleological Rescue Activities show 12,365 people were rescued due to hiking-related problems; of these people, 7,622 were injured and 491 were found dead [1]. These numbers increase when adding data from the Department of Health Activities, which recorded 10,000 Italians per year experiencing a variety of medical cases from mild to more severe, with symptoms including gastrointestinal issues, neurological complications, and even death. While we cannot directly solve the issues linked with the environmental dangers—that responsibility falls to the foresters of each country—we can help individuals focus more on their journey by warning them when they could find mushrooms. In this way, they are more aware of their surroundings and are focused on avoiding potential forest dangers. Using A-MYCO, a person can walk free of negative thoughts.

In order to achieve our goal, we trained the state-of-the-art object detection network YOLO11 [2] on a single-class dataset to generate boxes on a set of photos obtained from a specialised site, Mushroom Observer (MO) [3].
The system takes as input images from a detection camera, and if YOLO11 draws a bounding box on a certain frame of the video with enough confidence, another network, FasterViT, is used to provide a classification. The device communicates the species of identified mushrooms to the user through a speaker using Piper-TTS, a local neural text-to-speech system [4].
After an overview of the related works, we present the involved sites and the guidelines for the creation of the datasets, exploring a way to download new photos daily and update species' nomenclatures and, consequently, move already present images in the right directory.
Next, we talk more in detail about the neural network models used, justifying the preferred choices for the different trainings. The problems of data scarcity and unbalanced data are also addressed.
After that, we describe the image capture capability of our device and the audio output system.
Finally, the section dedicated to experimental results considers the measures to be implemented and the problems to be solved, opening the doors to future ideas and works outlined in the conclusions section.

II. Related Works

Following the "2018 FGCVx Fungi Classification Challenge," in which the main goal was to identify nearly 1,500 fungi species using over 100,000 images for model training [5], the works about mushrooms classification multiplied. It is notable, however, that the first studies on fungi recognition date back approximately twenty years, as evidenced by [6].
Instead of closely following the objectives of [5] or [6], we are going to detect and classify mushroom species based on camera frames. The kind of classification we want to consider is similar to the one reported in [7], but instead of relying solely on Convolutional Neural Networks (CNN) to extract mushroom features, we first employed YOLO11 as the discriminator and trained the network on datasets found in [8].
Numerous studies have been written regarding mushroom classification, and one of the latest is [9], which provides a comprehensive overview about the different techniques adopted previously. On the other hand, the majority of them

consider a binary edibility classification issue, e.g., [10] and the similar study [11]. Although the classes and the dimension of the dataset are entirely different compared to our work, the latter nevertheless is a relevant example of real-time mushroom detection using an embedded system. In addition, our work will consider species only, due to the non-general conventions adopted worldwide and due to the variability of the edibility label; in fact, the edibility of a mushroom is prone to changes in different countries and over time [12]. However, one can research details about the announced species and some edibility information may also be present in the spoken descriptions related to detected mushrooms.
Moreover, in this kind of problem, one should consider physical characteristics of a mushroom, and machine learning methods to analyse tabular data instead of images are therefore needed. This is demonstrated in [13] or [14], where characteristics such as the appearance of bruisings are considered, and in the challenge [15], where the best results were obtained using tree-based machine learning algorithms.
Again, in [9] some projects on species classification were reported, such as [16], and in analogous articles like [17], the dataset comes from a combination of various datasets, which brings misclassifications, e.g., *"Boletus badius"* is nowadays recognised as *"Imleria Badia"*, thus a mushroom from the *"Imleria"* genus could possibly be inside the *"Boletus"* one. For this reason, it is essential to rely on a single dynamic dataset as the one we consider in our work, which is an updated extension of the one used in [18] and an extended counterpart of [19].
Speaking of available applications for the identification of mushrooms, most of them are not open source, fall short in both the number of species covered and the level of accuracy required for real-time inference, or the offline mode requires a fee. Some examples are:

- **Picture Mushroom** [20], which uses multi-input photos to produce an output;
- **Mushroomizer** [21], a good real-time app for Android only but not open source, based on detection carried out using a smartphone while walking which can be risky;
- **Champignouf** [22], a valid free application, which however performs poorly during real-time scanning, as it primarily focuses on photo-based classification rather than detection;
- **ShroomID** [23] and **FungID** [24], which require the user to manually detect mushrooms.

Automatic mushroom detection prevents the user from being distracted by the dangers of the forest journey, avoiding an increase in risks related to hiking. This detection feature is mainly applied in agricultural harvesting; thus, the interest is wide, but species are very limited, as reported by [25], which exploits instance segmentation. However, we apply mushroom detection in the context of classification with a moving camera. Moreover, differently from this work, we relied on a more limited training apparatus, using an edge-inferencing device.
The dataset we built is possibly the hardest benchmark for the fungi kingdom available and also the most updated one, because deprecated scientific names were rejected in favour of the newly accepted ones, as described in III-B, and the methods and scale of the problem are truly unique. This underscores the difficulty of detecting and classifying all known mushroom species using only photos; biological analysis is required to reach a confidence level close to 100%. In fact, the most accurate technique for mushroom recognition is DNA analysis from a small sample, but this process requires a specialized machine, which is typically bulky.
It is important to note that the cited works make limited adjustments to achieve ideal frame conditions, such as accounting for variations in illumination, rotation, or more extensive image manipulations.

III. Methodologies

A. Deep Learning Models

YOLO11 is the last entry in the YOLO family, and it is a state-of-the-art (SOTA) model for object detection [2]. Unlike two-stage detectors (e.g., R-CNN), YOLO performs object detection in a single step, making it extremely fast. It simultaneously predicts multiple bounding boxes and their class probabilities in one evaluation, and it is designed for real-time applications; thus, its inference speed is useful considering the size of the MO dataset to be filtered. To obtain reliable bounding boxes on the MO images, we fine-tuned **YOLO11-l** on a mushroom dataset for object detection, adding $\sim 10\%$ images as background to reduce false positive biases.

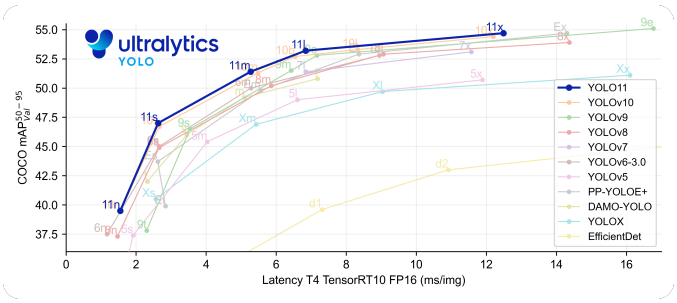


Fig. 1: Accuracy and latency of object detection on COCO across YOLO versions (measured on Nvidia T4) [2]

We choose to train a YOLO model due to the high-level training Ultralytics provides, in order to achieve good results without concentrating on the hyper-parametrization part. Moreover, it can be easily exported in the NCNN format, highly optimized for mobile/embedded platforms (such as the ARM architecture), and its structures and weights can be converted to INT8 and simplified with a single command [26]. In addition, its architecture has an attention mechanism that leads to more accurate detection, especially for smaller or partially occluded objects [27]. Combining these capabilities and light pruning techniques, the chosen model will perform a fast inference on the portable device.
FasterViT is a family of hybrid CNN-ViT (Convolutional Neural Network-Vision Transformer) models designed for

high image throughput in computer vision applications. It introduces a new self-attention mechanism that captures both short and long-range information by learning cross-window carrier tokens, particularly useful when performing fine-grained classification as in our problem. It achieves SOTA performance in terms of both accuracy and image throughput [28]. We considered FasterViT-0, the least demanding model across the family, with an ONNX format conversion code provided by the authors, followed by a simplification of the architecture and an NCNN conversion. Moreover, it can use rectangular images, so it is optimal to use the medians of the sides of the final dataset during training in order not to be influenced by outlier dimensions.

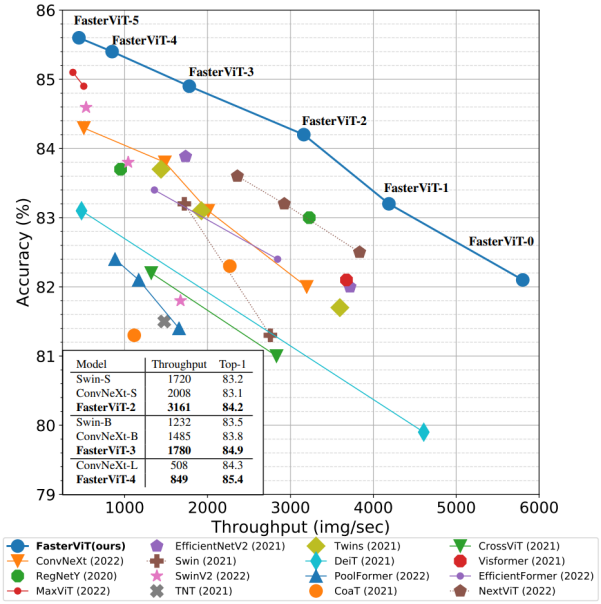


Fig. 2: FasterViT achieves a new SOTA Pareto-front in terms of Top-1 accuracy and throughput [28]

B. Dataset Creation and Workflow

A poorly constructed dataset is directly linked to inaccuracies in the resulting neural network model. To ensure consistency in species naming, we relied solely on MO, however, the option to scrape images from the whole web remains available[1].
MO is a collaborative mycology website started by Nathan Wilson in 2006 [30]. The key advantages of the site over others are:

- **Specialised Community**: MO is specifically focused on mushrooms, making it a great choice if you are primarily interested in mycology;
- **Detailed Identification**: MO offers more detailed and specialised identification processes, which can be beneficial for scientific research and accurate documentation;
- **Voting System**: MO has a robust voting system for identifications, allowing the community to help refine and correct identifications;
- **High Quality Images**: MO tends to have a higher proportion of high-quality images.

The GitHub page [31] of the website provides CSV files that are updated nightly and contain information about the more than 1.7 million images on the site, e.g., the species' name and the location of the mushroom. The downloaded images have a greater side of exactly 640 pixels, a rule-of-thumb value for the training of YOLO models.
We propose a path to recreate the dataset[2] considering the presence of information about deprecated denominations and synonyms:

1) Download the CSV files directly from the source;
2) Consider all the images on the site redistributing them in two main groups: *known* and *unknown* species;
3) Inside these groups, redistribute the images assuming as names the shortest non-deprecated name of the considered mushroom, among all the different names a species can have;
4) The different varieties, formae, subspecies and similar were distinguished, but considered as belonging to the referring species;
5) Discard the species belonging to misleading classifications such as "Mixed collections", and exclude images that only provide names at higher taxonomic levels without specifying the species;
6) Because different downloads of the CSV files can bring modifications on previously uploaded mushrooms on the site, such as genus changes, there was the need to create a script that corrects the names and the file locations of already downloaded images.

To make this extensive dataset compatible with object detection training, we had to create a separate dataset specifically tailored for detection.
There exist accessible datasets for projects about fungi, e.g., available in [32] and [8], and these are fundamental instruments for building models from scratch. In particular, we relied on the benchmark dataset used in [25], which is already split into training, validation, and test sets, transforming the coordinates for segmentation into the ones for detection while imposing the existence of a single class. We enriched the dataset using analogously [33] and [34] as mushroom images, and [35], [36], [37] as background, using pictures of fruits, flowers and faces. After doing that, we proceed through the following phases:

1) Perform Fine-Tuning (FT) of YOLO11-l on this dataset, monitoring the performances through the validation set;
2) Choose the threshold value of the confidence through the ROC curve of the test set;

[1]You may find the tutorial at [29]

[2]All the code of this project is available on our GitHub repository in https://github.com/Penzo00/AMYCO-Robot

3) Employ YOLO11-l to infere on the MO dataset with this threshold, detecting and discarding the corrupted images, and save the content of the boxes;
4) Perform an Exploratory Data Analysis (EDA) on the new dataset created by YOLO, to find the majority class and to inspect the distribution of the dataset;
5) Perform a non-invasive augmentation of the minority species using rotations by multiples of $\frac{\pi}{2}$, upside-down and right-left flips. To minimize the risk of transforming the side medians into a square shape, the $\frac{\pi}{2}$ rotations are applied last. Each operation is applied not only to the original images but also to the images already transformed by other operators. Furthermore, to preserve a real-case scenario, i.e., mushrooms with the stem below and cap above, we prioritized right-left flipping as the initial transformation.
   This augmentation step per class is stopped either if a species reaches the highest number of images in a class or if all the possible operations are applied within the same class;
6) Increase the number of photos of a symbolic 10% using a MixUp augmentation technique with two pair of images within the same class. This process is applied to all classes to prevent biases and promote fine-grained classification of niche particularities. We believe this approach provides an opportunity for the network to learn combined features within the same class, potentially mimicking the behavior of a multi-input image network, e.g., using separate images of stem, below the cap, and above the cap.
   In theory, considering all possible combinations and all classes, we reach a total number of images of:

$$N_f = \sum_i \frac{N_i!}{(2N_i - 2^{2-n_i})!} \tag{1}$$

   where N_i is the starting number of images of the species i, N_f the final number of the entire dataset, and n_i the number of symmetry operations performed in step 5 for the species i. In our case, $n_i = 3$ at most, allowing us to perform all planar transformations applicable to a 2D image. This is because symmetry and rotation operations on a 2D image belong to the dihedral group, which can be described by properly combining our three operations;
7) Train FasterViT-0 by choosing as rectangular input shape the medians of the created dataset.

At the end of this process, during inference, YOLO11-l will create boxes whenever a mushroom appears on the scene. The image inside the box will be transmitted to FasterViT-0, which will classify the species. If more than one mushroom is present on the scene, the one with the highest confidence will be considered.

To simplify the classification task during testing, we employed the MixUp augmentation technique using images captured two seconds apart to align the image distribution more closely with that used during training.

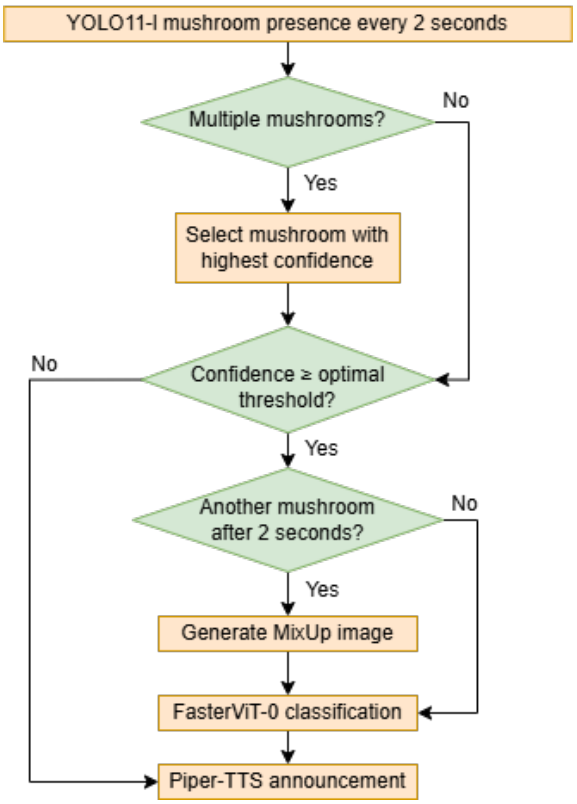


Fig. 3: Flowchart of the process



Fig. 4: Example of YOLO11 cropping ability used to increase the number of instances. In the image, Boletus edulis (var. grandedulis)

IV. Experimental Results and Discussion

A. Download of MO Images

The extimated time required to download the whole MO dataset is $\simeq$ 19 days, with approximately one image per second. The number of classes after YOLO11-l cleaning with optimal confidence is 12,952, starting from the original 14,355. We potentially extracted even more mushrooms from the raw images, as seen in Fig. 4.

B. Performance Evaluation

To evaluate performance, we used mean average precision at 50% Intersection over Union (IoU) (mAP@50) as the metric for YOLO training, and classification accuracy for FasterViT. **Classifying accuracy** is defined as the percentage of correctly

classified image types over the entire set of instances, as shown in the following equation:

$$Accuracy = \frac{Correct\ classifications}{Number\ of\ entire\ set} \tag{2}$$

Instead, **mAP@50** is a specific metric that calculates the average precision of the model at an IoU threshold of 50%. This means that for a prediction to be considered correct, the predicted bounding box must overlap with the ground truth box by at least 50%, and it is defined as:

$$mAP = \frac{1}{N}\sum_{i=1}^{N} AP_i \tag{3}$$

where N is the total number of classes (one in our case).

C. YOLO11 Training

We trained YOLO11-l for 150 epochs with a batch size of 16, using the AdamW optimizer. The initial and final learning rates were set to $5 \cdot 10^{-4}$ and $5 \cdot 10^{-5}$ respectively, following a cosine learning rate schedule. For augmentations, we used the default settings of [2], keeping a probability of 100% for mosaic segmentation.

As we can see in Fig. 5, training does not show a nervous behaviour. In validation we achieved mAP@50 of more than 0.86 using the simplified and optimized INT8 and NCNN converted model, compared to the 0.89 of the original model.

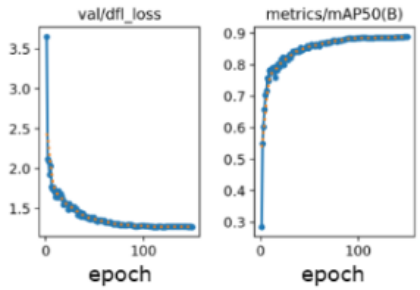


Fig. 5: YOLO11-l metrics in validation: Distribution Focal Loss and mAP are to be minimized and maximized respectively

		True		
		Mushroom	Background	Total
Predicted	Mushroom	3355	701	4056
	Background	658	-	658
	Total	4013	701	4714

		True		
		Mushroom	Background	Total
Predicted	Mushroom	2955	206	3161
	Background	1058	-	1058
	Total	4013	206	4219

Table I: Confusion matrices without (above) and with (below) the best threshold

We evaluated the optimal threshold value using a ROC curve and calculated the confusion matrices. In Table I, the sum of false positives and negatives with a suboptimal threshold is 1,447, compared to 1,264 using the suggested value, consolidating it as the better option.

It is noted that the confidence suggested by Fig. 6 was not enough to detect hard scenarios as in Fig. 7 on the right. However, one can improve the YOLO step as needed.

We propose two modes: **detection mode**, where the user sets a confidence threshold below the suggested value, and **classification mode**, where the threshold is set above the suggested value. For detection mode, we suggest a confidence higher than 0.024.

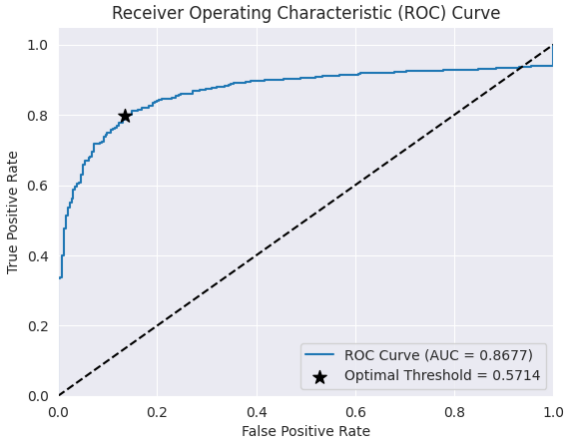


Fig. 6: ROC curve of YOLO11-l constructed using the test set



Fig. 7: Real-case scenarios using YOLO11

D. FasterViT Training

We chose to set $n = 3$ (in sequence: flip left-right, rotate by π and by $\frac{\pi}{2}$) in formula 1. Then, we enriched each class by 10% using a random intra-class MixUp augmentation. The final medians of the image sizes were **192x192**. We split the images in training and validation set (80%-20% division) and, finally, we deleted the classes with less than 1.000 images in training set, ending up with 1.822 classes, thus creating the **MO1822** dataset.

We trained opting for a standard image size of **224x224** using Lion optimizer, following the suggestions in [38]. In particular, we decided to implement:

- Batch size of 128;
- Learning rate with cosine decay from $2.5 \cdot 10^{-5}$ to 0;
- $\beta_1 = 0.95$ and $\beta_2 = 0.98$ to mitigate instabilities;
- No training-time augmentation;
- Weight decay of 0.4.

Concerning the network, we deleted the last 3 layers of FasterViT-0 and unfreezed all other layers except for the batch normalizations. Our classification head consisted of the following:

- Global average pooling layer;
- Dense layer with 2,048 neurons utilizing the orthogonal regularizer with factor 10^{-3} and *swish* as activation function, followed by a batch normalization;
- Another dense layer with 1,024 neurons, *swish* activation, and both L_1 and L_2 regularizers with factors of 10^{-3}, followed by a batch normalization and a dropout of 0.2;
- Output layer with 1,822 neurons, with *softmax* activation.

The loss function for our problem was ***Sparse Categorical Crossentropy***, and the class weights during the training were computed using the *sklearn* library. No other class balancing methods consisting in the creation of surrogate photos were implemented for a main reason: the amount of data would have been unsustainable for our time and resources, i.e., $24,283 \cdot 1,822 = 44,243,626$ images. We trained the model for 10 epochs with 7,007,225 training and 1,752,717 validation pictures.

The ideal augmented image is shown in Fig. 8, where we can observe the cap, stem, roots, and gills of the mushroom simultaneously, but inter-species MixUp is also applied by default.



Fig. 8: Ideal augmented image using the MixUp technique

Despite the esiguous number of epochs and the absence of hyper-parameter tuning, we achieved a **Top-1 and Top-5 accuracies** in validation of almost **97.62%** and **99.39%**, respectively (Fig. 9). We further tested the models with 9 representative mushroom photos, labeled by Gruppo Micologico Milanese [39], reaching 100% Top-1 accuracy. Two of these photos are in Fig. 11.

E. Piper-TTS Voices

The species' names are in Latin, while the descriptions are in English, so we opted for a female Italian voice for the names

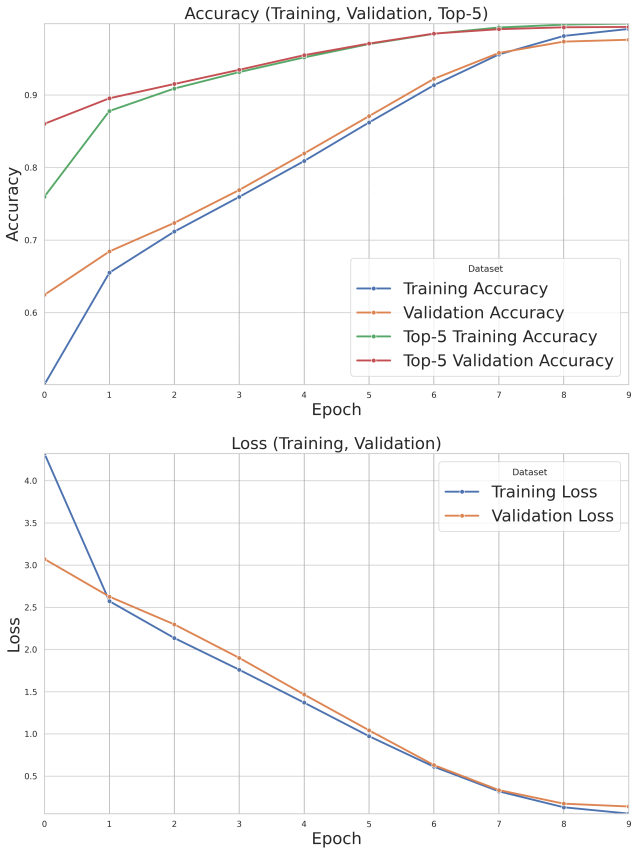


Fig. 9: Monitoring of accuracies and losses of FasterViT-0

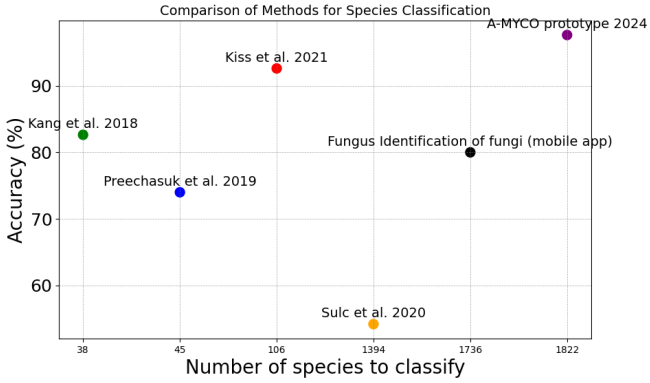


Fig. 10: Comparison of accuracies reached in different works

[40] and a female British voice for the descriptions [41].

F. Prototype

We completed the project by processing everything on a Raspberry Pi 400, which is based on a Broadcom BCM2711 quad-core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8 GHz processor, with 4 GB of RAM [42]. By exploiting its multiple ports, we connected a desktop webcam, a speaker, and a monitor to test the functioning of the system. Our goal was to track the resource usage of the models on the hardware and assess

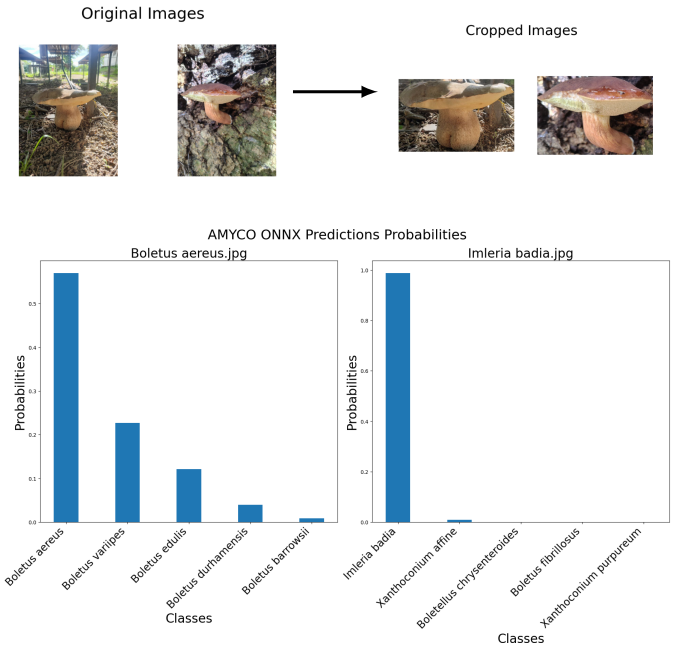


Fig. 11: Top-5 species in two of the final test photos. These species are hard to distinguish, but the network predicts distinct classes for each image

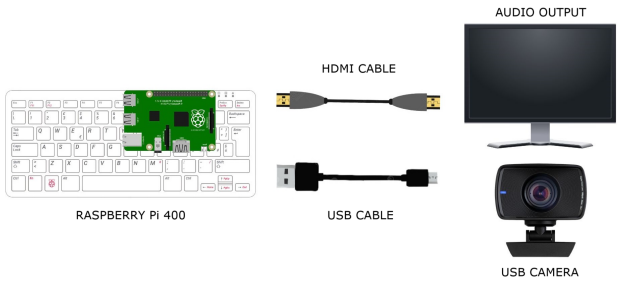


Fig. 12: Scheme of the testing device: a Raspberry Pi 400. The images are taken by an USB camera, while the audio is emitted by another device, e.g., a TV, connected through an HDMI cable

their potential for installation on a smart device. We found that the first step of mushroom identification using YOLO11-l NCNN converted operates with low enough RAM usage. However, the step of mushroom classification with FasterViT, with its powerful features for fine-grained classification, incurs a higher RAM cost, feasible thanks to the ONNX conversion. Our initial goal was to use these models on a wearable device capable of scanning the surrounding area and providing dynamic responses. We aimed to use the Raspberry Pi Zero 2W due to its lightness, compactness, and features, such as the dedicated compact camera module for real-time image detection, and the Bluetooth antenna for connecting wireless peripherals, like the speaker needed to announce the detected mushroom's name. However, for Raspberry Pi Zero 2W a further optimization is needed, e.g., by considering pruning techniques, due to its RAM of 512 MB [43].

In the end, we decided to rely on the future implementation of microprocessors, as the trend is moving towards the integration of Neural Processing Units (NPUs) designed to run models efficiently. By prioritizing core functionality, we refined the design to strike a balance between being lightweight, precise, and efficient. This approach ensures broader applicability, making the system adaptable to various hardware environments rather than being limited to a specific setup.

V. Conclusions

Although we achieved high validation accuracy, every aspect of this work can be further improved. This includes training the detection network with a wider variety of mushrooms, considering different phases within the same species' life cycle, and potentially replacing YOLO with transformer-based detection networks, such as DETR, trained on higher-resolution images. Subsequently, we could use this improved detection model to filter the MO dataset, followed by a CNN to distinguish between good and poor-quality photos, resulting in a refined dataset containing only feature-rich images.

We used the Lion optimizer, which delivers the best results when augmentation is minimal. This is a significant drawback of this optimizer. Thus, in order to maintain its efficiency, we could preprocess the images using a gyroscope to correct for rotation caused by camera movement, along with illumination enhancements to address various environmental challenges that cameras may face. Additionally, we addressed data scarcity by performing rotations of $\frac{\pi}{2}$. However, this should be avoided to better exploit rectangular sizes with a ViT architecture, so the use of Generative Adversarial Networks (GANs) could be explored to generate synthetic images of the least represented species.

The greatest improvement will be achieved by fully leveraging certain distinctive features of mushrooms, that is, by exploiting the geographical position using a GPS, accompanied by the physical characteristics described in [13].

In the future, analysing small portions of mushroom DNA on-site, along with the previously mentioned information, could finally pave the way for advanced versions of A-MYCO assistants.

References

[1] CNSAS, "Disponibili i dati 2023 delle attività del soccorso alpino e speleologico," online, April 2024, https://news.cnsas.it/disponibili-i-dati-del-2023-delle-attivita-del-soccorso-alpino-e-speleologico/.

[2] G. Jocher and J. Qiu, "Ultralytics yolo11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[3] N. Wilson, J. Hollinger *et al.*, "Mushroom observer," 2006–present, accessed: 2024-10-15. [Online]. Available: https://mushroomobserver.org

[4] Rhasspy, "Piper: A fast, local neural text to speech system," 2024, accessed: July 12, 2026. [Online]. Available: https://github.com/rhasspy/piper

[5] Visipedia, "Fgvcx fungi classification challenge," https://github.com/visipedia/fgvcx_fungi_comp, accessed: 2024-10-15.

[6] P. Perner, S. Jänichen, and H. Perner, "Case-based object recognition for airborne fungi recognition," *Artificial Intelligence in Medicine*, vol. 36, no. 2, pp. 137–157, 2006, case-based reasoning in the health

sciences. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0933365705000837
[7] N. Kiss and L. Czuni, "Mushroom image classification with cnns: A case-study of different learning strategies," 09 2021, pp. 165–170.
[8] "Roboflow mushroom dataset search results," n.d., accessed: 2024-10-01. [Online]. Available: https://universe.roboflow.com/search?q=mushroom+object+detection+images%3E5000&advanced=true
[9] E. Özbay, F. A. Özbay, and F. S. Gharehchopogh, "Visualization and classification of mushroom species with multi-feature fusion of metaheuristics-based convolutional neural network model," *Applied Soft Computing*, vol. 164, p. 111936, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494624007105
[10] J. Preechasuk, O. Chaowalit, F. Pensiri, and P. Visutsak, "Image analysis of mushroom types classification by convolution neural networks," 12 2019, pp. 82–88.
[11] T. Bouganssa, A. Salbi, S. Aarabi, A. Lasfar, and A. E. Afia, "Recognition of mushrooms and classification of edible and toxic families using hardware implementation of cnn algorithms on an embedded system," *Research Journal of Pharmacy and Technology*, vol. 17, no. 2, pp. 860–866, 2024. [Online]. Available: https://rjptonline.org/AbstractView.aspx?PID=2024-17-2-61
[12] N. Sitta, P. Davoli, M. Floriani, and E. Suriano, *Guida ragionata alla commestibilità dei funghi*, 09 2021.
[13] "Mushroom," UCI Machine Learning Repository, 1981, DOI: https://doi.org/10.24432/C5959T.
[14] D. Wagner, D. Heider, and G. Hattab, "Mushroom data creation, curation, and simulation to support classification tasks," *Scientific Reports*, vol. 11, no. 1, p. 8134, 2021. [Online]. Available: https://doi.org/10.1038/s41598-021-87602-3
[15] A. C. Walter Reade, "Binary prediction of poisonous mushrooms," 2024. [Online]. Available: https://kaggle.com/competitions/playground-series-s4e8
[16] E. Kang, Y. Han, and I.-S. Oh, "Mushroom image recognition using convolutional neural network and transfer learning," *KIISE Transactions on Computing Practices*, vol. 24, no. 1, pp. 53–57, 2018. [Online]. Available: http://dx.doi.org/10.5626/KTCP.2018.24.1.53
[17] Y. Kang, Y. Tang, J. Jiang, and Y. Jia, "Mushroom image classification based on improved mobilenetv2," in *Proceedings of the 2022 5th International Conference on Telecommunications and Communication Engineering*, ser. ICTCE '22. New York, NY, USA: Association for Computing Machinery, 2023, p. 232–237. [Online]. Available: https://doi.org/10.1145/3577065.3577107
[18] Y. Peng, Y. Xu, J. Shi, and S. Jiang, "Wild mushroom classification based on improved mobilevit deep learning," *Applied Sciences*, vol. 13, no. 8, 2023. [Online]. Available: https://www.mdpi.com/2076-3417/13/8/4680
[19] L. Picek, K. Janouskova, M. Sulc, and J. Matas, "FungiTastic: A multi-modal dataset and benchmark for image categorization," *arXiv preprint arXiv:2408.13632*, 2023, available at: https://doi.org/10.48550/arXiv.2408.13632.
[20] "Picture mushroom app," n.d., accessed: 2024-10-01. [Online]. Available: https://picturemushroom.com/it/app#:~:text=Prova%20l'app%20Picture%20Mushroom%20sul%20tu
[21] "Mushroomizer: Mushroom identification app," n.d., accessed: 2024-10-01. [Online]. Available: https://play.google.com/store/apps/details/Mushroomizer?id=my.tensorflow.lite.examples.classification
[22] "Champignouf: App per l'identificazione dei funghi," n.d., accessed: 2024-10-01. [Online]. Available: https://play.google.com/store/apps/details?id=com.pingou.champignouf&hl=it
[23] "Shroom.id: Mushroom identification tool," n.d., accessed: 2024-10-01. [Online]. Available: https://shroom.id/
[24] "Fungid: Accurate app for mushroom identification," n.d., accessed: 2024-10-01. [Online]. Available: https://fungid.app/#:~:text=The%20most%20accurate%20app%20for%20identifying
[25] A. Zakeri, M. Fawakherji, J. Kang, B. Koirala, V. Balan, W. Zhu, D. Benhaddou, and F. A. Merchant, "M18k: A comprehensive rgb-d dataset and benchmark for mushroom detection and instance segmentation," *ArXiv*, vol. abs/2407.11275, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:271218246
[26] Ultralytics, "Use ncnn on raspberry pi," 2024, accessed: July 12, 2026. [Online]. Available: https://docs.ultralytics.com/guides/raspberry-pi/#use-ncnn-on-raspberry-pi
[27] R. Khanam and M. Hussain, "Yolov11: An overview of the key architectural enhancements," 2024. [Online]. Available: https://arxiv.org/abs/2410.17725
[28] A. Hatamizadeh, G. Heinrich, H. Yin, A. Tao, J. M. Alvarez, J. Kautz, and P. Molchanov, "Fastervit: Fast vision transformers with hierarchical attention," 2024. [Online]. Available: https://arxiv.org/abs/2306.06189
[29] Bam098, "Creating mushroom dataset: Update," https://bam098.github.io/blog/dataset/image%20classification/fastai/cnn/duckduckgo/2021/01/02/creating-mushroom-dataset-update.html, 2021, accessed: July 12, 2026.
[30] "Mushroom observer," https://en.wikipedia.org/wiki/Mushroom_Observer, accessed: July 12, 2026.
[31] "Mushroom observer api," https://github.com/MushroomObserver/mushroom-observer/blob/main/README_API.md#csv-files, accessed: July 12, 2026.
[32] D. Tan, "Mushroom learning: A machine learning project for mushroom classification," https://github.com/DSP-Tan/mushroom_learning, accessed: 2024-10-15.
[33] P. N. Padang, "Tugas akhir dataset," https://universe.roboflow.com/politeknik-negeri-padang-qljai/tugas-akhir-qrqzr, aug 2023, visited on 2024-11-28. [Online]. Available: https://universe.roboflow.com/politeknik-negeri-padang-qljai/tugas-akhir-qrqzr
[34] Free, "Fly agaric dataset," https://universe.roboflow.com/free-ha4h2/fly-agaric, jan 2024, visited on 2024-11-28. [Online]. Available: https://universe.roboflow.com/free-ha4h2/fly-agaric
[35] pp, "mangkud dataset," https://universe.roboflow.com/pp-qdrvp/mangkud, oct 2022, visited on 2024-11-28. [Online]. Available: https://universe.roboflow.com/pp-qdrvp/mangkud
[36] Eyes, "eyes dataset," https://universe.roboflow.com/eyes-blh5y/eyes-vltyc, mar 2024, visited on 2024-11-28. [Online]. Available: https://universe.roboflow.com/eyes-blh5y/eyes-vltyc
[37] myfirst, "flower-yolov5 dataset," https://universe.roboflow.com/myfirst-yzoar/flower-yolov5-ucpnv, nov 2022, visited on 2024-11-28. [Online]. Available: https://universe.roboflow.com/myfirst-yzoar/flower-yolov5-ucpnv
[38] X. Chen, C. Liang, D. Huang, E. Real, K. Wang, Y. Liu, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu, and Q. V. Le, "Symbolic discovery of optimization algorithms," 2023. [Online]. Available: https://arxiv.org/abs/2302.06675
[39] Gruppo Micologico Milanese, "Gruppo micologico milanese," 2024, accessed: 2024-12-08. [Online]. Available: http://www.gruppomicologicomilanese.it/
[40] M. Hansen and contributors, "Piper voices: Paola (it_it)," 2024, accessed: 2024-12-05. [Online]. Available: https://huggingface.co/rhasspy/piper-voices/tree/main/it/it_IT/paola/medium
[41] ——, "Piper voices: Cori (en_gb)," 2024, accessed: 2024-12-05. [Online]. Available: https://huggingface.co/rhasspy/piper-voices/tree/main/en/en_GB/cori/high
[42] R. P. Foundation, "Raspberry pi 400 product brief," 2020, accessed: 2024-12-13. [Online]. Available: https://datasheets.raspberrypi.com/rpi400/raspberry-pi-400-product-brief.pdf
[43] ——, "Raspberry pi zero 2 w product brief," 2021, accessed: 2024-12-13. [Online]. Available: https://datasheets.raspberrypi.com/rpizero2/raspberry-pi-zero-2-w-product-brief.pdf